

```
"""
```

Scrapere pentru OLX, Storia si Publi24.

IMPORTANT: aceste site-uri isi pot schimba structura HTML oricand, ceea ce poate "rupe" un scraper existent. De asta fiecare functie are un fallback si un mod --debug (vezi main.py) care iti arata cate rezultate a gasit fiecare metoda de extractie, ca sa poti depana rapid daca ceva nu merge.

OLX si Storia au sisteme anti-bot (rate limiting / verificari de trafic).

Daca rulezi scriptul foarte des sau de pe un server cloud cu IP "suspect", e posibil sa primești raspunsuri goale sau erori HTTP 403. Solutii:

- ruleaza la interval de 5-10 minute, nu mai des
- foloseste headerele de mai jos (simuleaza un browser normal)
- daca tot esueaza constant, ia in calcul pachetul `cloudscraper` in loc de `requests` pentru acele site-uri specifice

```
"""
```

```
import re  
import json
```

```

import requests
from bs4 import BeautifulSoup

HEADERS = {
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0;
Win64; x64) AppleWebKit/537.36 "
        "(KHTML, like Gecko)
Chrome/124.0.0.0 Safari/537.36"
    ),
    "Accept": (
        "text/html,application/xhtml+xml,application/xml;q=0.9,"
        "image/avif,image/webp,*/*;q=0.8"
    ),
    "Accept-Language": "ro-R0,ro;q=0.9,en-US;q=0.8,en;q=0.7",
    "Accept-Encoding": "gzip, deflate, br",
    "Connection": "keep-alive",
    "Upgrade-Insecure-Requests": "1",
    "Sec-Fetch-Dest": "document",
    "Sec-Fetch-Mode": "navigate",
    "Sec-Fetch-Site": "none",
    "Sec-Fetch-User": "?1",
    "Cache-Control": "max-age=0",
}

# Folosim o singura sesiune (pastreaza
# cookie-urile intre cereri, ca un
# browser normal) in loc de cereri complet
# independente.
# Folosim o singura sesiune (pastreaza

```

```

cookie-urile intre cereri, ca un
# browser normal) in loc de cereri complet
independente.
_session = requests.Session()
_session.headers.update(HEADERS)
_visited_homepages = set()

def _fetch(url, source="?", debug=False):
    try:
        # Vizitam intai pagina principala a
        site-ului, ca sa primim
        # cookie-urile initiale (unele
        sisteme anti-bot verifica asta),
        # apoi cerem pagina reala cu
        Referer setat spre site.
        parts = url.split("/")
        homepage = f"
{parts[0]}://{parts[2]}/"
        if homepage not in
        _visited_homepages:
            try:
                _session.get(homepage,
                timeout=15)
            except
            requests.RequestException:
                pass # daca esueaza
                homepage-ul, incercam oricum pagina reala
        _visited_homepages.add(homepage)

        resp = _session.get(url, headers=
        {"Referer": homepage}, timeout=20)
        except requests.RequestException as e:

```

```

        print(f"[{source}] EROARE de retea
la {url}: {e}")
        raise

    if debug:
        print(f"[{source}] HTTP
{resp.status_code}, {len(resp.text)}
caractere primite de la {url}")

    if resp.status_code != 200:
        # Afisam un fragment din raspuns -
deseori aici scrie exact ce s-a
        # intamplat (pagina de verificare
anti-bot, captcha, etc.)
        snippet =
resp.text[:300].replace("\n", " ")
        print(f"[{source}] HTTP
{resp.status_code} neasteptat. Fragment din
raspuns: {snippet!r}")

    resp.raise_for_status()
    return resp.text

# -----
# -----
# OLX
# -----
# -----

def scrape_olx(url=None, debug=False):
    from config import SEARCH_URLS
    url = url or SEARCH_URLS["olx"]

```

```

        html = _fetch(url, source="OLX",
debug=debug)
        soup = BeautifulSoup(html,
"html.parser")
        results = []

        cards = soup.select('[data-cy="l-
card"]')
        if debug:
            print(f"[OLX] {len(cards)} carduri
gasite cu selectorul principal")

        for card in cards:
            link_tag = card.find("a",
href=True)
            if not link_tag:
                continue
            href = link_tag["href"]
            if href.startswith("/"):
                href = "https://www.olx.ro" +
href

            # ID-ul OLX apare de obicei in URL
ca "...-IDxxxxxxx.html"
            m = re.search(r"-ID([a-zA-Z0-
9]+)\.html", href)
            ad_id = m.group(1) if m else href

            title_tag = card.select_one('[data-
testid="ad-title"]') or card.find(["h4",
"h6"])
            title =
title_tag.get_text(strip=True) if title_tag

```

```

else "Anunt fara titlu"

        price_tag = card.select_one('[data-
testid="ad-price"]')
        price =
price_tag.get_text(strip=True) if price_tag
else "N/A"

        # OLX marcheaza anunturile de la
        conturi de tip "Firma" cu o
        # eticheta vizibila pe card
        (agentiile imobiliare posteaza de pe
        # conturi de firma). Filtrul din
        URL (private_business=private)
        # ar trebui sa elimine deja
        majoritatea, dar verificam si aici
        # ca rezerva, in caz ca parametrul
        nu e respectat de site.
        card_text = card.get_text(" ",
strip=True)
        is_business = "Firmă" in card_text
or "Firma" in card_text

        results.append({
            "id": f"olx_{ad_id}",
            "title": title,
            "price": price,
            "url": href,
            "source": "OLX",
            "is_agency": is_business,
        })

if not results and debug:
    print("[OLX] Niciun rezultat –

```

```

structura paginii s-a putut schimba. "
    "Ruleaza cu --debug si
verifica manual pagina in browser.")

    return results

# -----
# Storia (site Next.js – datele sunt de
# obicei intr-un JSON __NEXT_DATA__)
# -----

def _storia_is_agency(item):
    """
    Cea mai buna estimare, din campuri JSON
    cunoscute de la site-uri similare
    (Otodom/Storia). Structura reala se
    poate schimba – ruleaza cu --debug
    ca sa vezi cheile disponibile si
    ajusteaza aici daca e nevoie.
    """
    if item.get("isPrivateOwner") is True:
        return False
    if item.get("isPrivateOwner") is False:
        return True

    advertiser_type = str(
        item.get("advertiserType") or
        item.get("advertiser_type") or ""
    ).lower()
    if advertiser_type:
        return advertiser_type != "private"

```

```

        # Daca anuntul are un nume de
        agentie/dezvoltator atasat, il consideram
        agentie.
        agency = item.get("agency") or
        item.get("agencyName") or
        item.get("developerName")
        if agency:
            return True

        # Necunoscut -> nu excludem (mai bine
        trimitem in plus decat sa ratam
        # un anunt de la proprietar din cauza
        unui camp gresit ghicit).
        return False

def scrape_storia(url=None, debug=False):
    from config import SEARCH_URLS
    url = url or SEARCH_URLS["storia"]

    html = _fetch(url, source="Storia",
    debug=debug)
    soup = BeautifulSoup(html,
    "html.parser")
    results = []

    script = soup.find("script",
    id="__NEXT_DATA__")
    if script and script.string:
        try:
            data =
            json.loads(script.string)
            items = (

```



```

        data.get("props", {})
            .get("pageProps", {})
            .get("data", {})
            .get("searchAds", {})
            .get("items", [])
    )
    if debug:
        print(f"[Storia]
{len(items)} anunturi gasite in JSON
__NEXT_DATA__")
        if items:
            print(f"[Storia] Chei
disponibile in primul anunt (util pentru "
                f"ajustarea
filtrului proprietar/agentie):
{list(items[0].keys())}")
            for it in items:
                slug = it.get("slug", "")
                ad_id = it.get("id", slug)
                title = it.get("title",
"Anunt fara titlu")
                price_obj =
it.get("totalPrice") or {}
                price = f"
{price_obj.get('value', '?')}
{price_obj.get('currency', '')}".strip()
                ad_url =
f"https://www.storia.ro/ro/oferta/{slug}"
                results.append({
                    "id":
f"storia_{ad_id}",
                    "title": title,
                    "price": price,
                    "url": ad_url,

```

```

        "source": "Storia",
        "is_agency":
_storia_is_agency(it),
    })
    if results:
        return results
    except Exception as e:
        if debug:
            print(f"[Storia] Parsarea
JSON a esuat ({e}), trec pe fallback HTML")

    # Fallback: caută direct linkurile
    către oferte în HTML
    if debug:
        print("[Storia] Folosesc fallback
        bazat pe linkuri <a
        href='/ro/oferta/...'>")
        seen_hrefs = set()
        for a in soup.find_all("a", href=True):
            href = a["href"]
            if "/ro/oferta/" not in href or
href in seen_hrefs:
                continue
            seen_hrefs.add(href)
            full_url = href if
href.startswith("http") else
"https://www.storia.ro" + href
            title = a.get_text(strip=True) or
            "Anunt fara titlu"
            results.append({
                "id": f"storia_{href}",
                "title": title,
                "price": "N/A",
                "url": full_url,

```

```
        "source": "Storia",
        "is_agency": None, #
necunoscut in modul fallback -> nu filtram
dupa asta
    })
    return results
```

```
# -----
-----
# Publi24
# -----
-----
```

```
# Cuvinte care, gasite in descrierea unui
anunt, indica aproape sigur ca
# a fost postat de proprietar direct
(verificat manual pe anunturi reale
# de pe Publi24 – proprietarii scriu
aproape mereu unul din tiparele astea).
_PUBLI24_OWNER_HINTS = [
    "proprietar", "particular,",
    "particular ", "direct proprietar",
    "fara agentie", "fără agenție", "fara
agentii", "fără agenții",
]
```

```
# Tipare care indica o agentie: numele
firmei urmat de o formula de genul
# "va propune"/"va oferi", sau mentionarea
explicita a cuvantului agentie.
_PUBLI24_AGENCY_PATTERNS = [
    r"v[ăa]\s+propun[eă]",
    r"v[ăa]\s+ofer[ăa]",
```

```

r"v[ăa]\s+prezint[ăa]",
    r"agen[țt]i[ae]\s+imobiliar",
r"imobiliare\s+v[ăa]",
]

def _publi24_card_text(a_tag):
    """Incearca sa gaseasca textul complet
    al cardului (titlu + descriere),
    urcand prin parintii tagului <a> pana
    gaseste un bloc cu text suficient."""
    node = a_tag
    for _ in range(6):
        if node.parent is None:
            break
        node = node.parent
        text = node.get_text(" ",
strip=True)
        if len(text) > 120:
            return text
    return a_tag.get_text(" ", strip=True)

def _publi24_is_agency(card_text):
    text_lower = card_text.lower()
    if any(hint in text_lower for hint in
_PUBLI24_OWNER_HINTS):
        return False
    for pat in _PUBLI24_AGENCY_PATTERNS:
        if re.search(pat, text_lower):
            return True
    # Niciun semnal clar -> nu excludem
    (mai bine trimitem in plus decat
    # sa ratam un anunt real de la un

```

```

proprietar).
    return False

def scrape_publi24(url=None, debug=False):
    from config import SEARCH_URLS
    url = url or SEARCH_URLS["publi24"]

    html = _fetch(url, source="Publi24",
debug=debug)
    soup = BeautifulSoup(html,
"html.parser")
    results = []
    seen_hrefs = set()
    agency_count = 0

    for a in soup.find_all("a", href=True):
        href = a["href"]
        if "/anunt/" not in href or not
href.endswith(".html") or href in
seen_hrefs:
            continue
        seen_hrefs.add(href)

        title = a.get_text(strip=True)
        if not title:
            # unele linkuri sunt doar pe
            imagine, fara text -> le sarim
            continue

        full_url = href if
href.startswith("http") else
"https://www.publi24.ro" + href
        m = re.search(r"/([a-z0-9]

```

```

{20,})\}.html", href)
    ad_id = m.group(1) if m else href

    card_text = _publi24_card_text(a)
    is_agency =
_publi24_is_agency(card_text)
    if is_agency:
        agency_count += 1

    results.append({
        "id": f"publi24_{ad_id}",
        "title": title,
        "price": "N/A",
        "url": full_url,
        "source": "Publi24",
        "is_agency": is_agency,
    })

    if debug:
        print(f"[Publi24] {len(results)}
anunturi gasite, "
            f"{agency_count} marcate ca
agentie (best-effort)")
        if not results:
            snippet =
html[:500].replace("\n", " ")
            print(f"[Publi24] 0 rezultate
desi HTTP 200 – fragment din pagina
primita: {snippet!r}")

    return results

```

